

Shinde Procurement — End-to-End Data Flow Test Plan

Purpose: verify that every core action performed in the mobile app actually lands correctly in the Postgres database, in the right tables, with the right relationships. Each test case = steps to perform in the app + a SQL query to run right after, to confirm the backend did what it should.

Run these in order — later modules (Cart, Purchase Order) depend on data created in earlier ones (Vendor, Product, Variant, Size).

Tip: run `CALL reset_shinde_procurement_defaults();` first so you're testing against a clean, known state.

1. Vendor

TC-01: Create a vendor

App steps: Vendor list screen → Add Vendor → fill name, code, contact person, mobile, email, GST/PAN → Save.

Expected: New row appears in `vendor`, status `active`.

sql

```
SELECT vendor_id, vendor_name, vendor_code, status, created_date
FROM vendor
ORDER BY created_date DESC
LIMIT 1;
```

✓ Pass if: one row returned, `status = 'active'`, `vendor_name` matches what you typed.

TC-02: Edit a vendor (partial update doesn't wipe other fields)

App steps: Open the vendor from TC-01 → Edit → change only the mobile number → Save.

sql

```
SELECT vendor_name, vendor_code, contact_person, mobile_number, email, updated_date
FROM vendor
WHERE vendor_id = <id_from_TC01>;
```

✓ Pass if: `mobile_number` changed, every other field (name, code, contact_person, email) is **unchanged** — this specifically guards against the old full-replace bug mentioned in `Vendor.update()`.

TC-03: Deactivate a vendor

App steps: Vendor list → toggle vendor to inactive.

sql

```
SELECT vendor_id, status FROM vendor WHERE vendor_id = <id_from_TC01>;
```

✓ Pass if: `status = 'inactive'`.

2. Product

TC-04: Add a product under a vendor

App steps: Open vendor workspace → Product Catalogue → Add Product → enter product code, name, price, upload photo → Save.

sql

```
SELECT product_id, vendor_id, v_prod_id, product_name, price, photo_url, status  
FROM product  
WHERE vendor_id = <id_from_TC01>  
ORDER BY created_date DESC  
LIMIT 1;
```

✓ Pass if: row exists, `vendor_id` matches, `status = 'active'`, `photo_url` is populated (check the file also exists under `uploads/products/` on the backend).

TC-05: Duplicate product code under the same vendor is rejected

App steps: Try adding another product with the same `v_prod_id` for the same vendor.

sql

```
SELECT COUNT(*) FROM product  
WHERE vendor_id = <id_from_TC01> AND v_prod_id = '<same_code>';
```

✓ Pass if: app shows an error and this count stays `1` (enforced by `product_vendor_id_v_prod_id_key` unique constraint).

3. Variant

TC-06: Add a variant with free-text category/gender/color/pattern

App steps: Open the product from TC-04 → Add Variant → type a **new** category name, gender, color, pattern not already in the master lists → Save.

```
sql

-- confirm the variant was created
SELECT v.variant_id, v.product_id, c.category_name, g.gender_name, p.pattern_name,
FROM variant v
LEFT JOIN category c ON v.category_id = c.category_id
LEFT JOIN gender g ON v.gender_id = g.gender_id
LEFT JOIN pattern p ON v.pattern_id = p.pattern_id
LEFT JOIN color col ON v.color_id = col.color_id
WHERE v.product_id = <id_from_TC04>;

-- confirm find_or_create actually inserted new lookup rows instead of erroring
SELECT * FROM category WHERE category_name = '<the new category you typed>';
SELECT * FROM gender WHERE gender_name = '<the new gender you typed>';
```

✅ Pass if: variant row exists with all four lookup IDs populated, and each new lookup value now exists as its own row (case-insensitive match, no duplicates if you re-add the same name — try re-adding the same category name in a second variant and confirm `category` still has only one row for it).

TC-07: Deactivate a variant

```
sql

SELECT variant_id, status FROM variant WHERE variant_id = <id_from_TC06>;
```

✅ Pass after toggling inactive in-app: `status = 'inactive'`.

4. Sizes

TC-08: Assign sizes to a variant

App steps: Open the variant from TC-06 → Sizes screen → select sizes 6, 7, 8 → Save.

```
sql
```

```
SELECT vs.variant_id, s.size_value, vs.status
FROM variant_size vs
JOIN size s ON s.size_id = vs.size_id
WHERE vs.variant_id = <id_from_TC06>
ORDER BY s.size_value;
```

✓ Pass if: exactly rows for 6, 7, 8 exist, all (`status = 'active'`).

TC-09: Deselect a size (sync, not just insert)

App steps: Reopen Sizes screen for the same variant, uncheck size 7, save.

sql

```
SELECT s.size_value
FROM variant_size vs
JOIN size s ON s.size_id = vs.size_id
WHERE vs.variant_id = <id_from_TC06>;
```

✓ Pass if: only 6 and 8 remain — confirms (`VariantSize.assign_sizes()`) actually deletes deselected sizes instead of just adding.

TC-10: Re-saving the same sizes doesn't error or duplicate

App steps: Reopen Sizes screen, save without changing anything.

sql

```
SELECT variant_id, size_id, COUNT(*)
FROM variant_size
WHERE variant_id = <id_from_TC06>
GROUP BY variant_id, size_id
HAVING COUNT(*) > 1;
```

✓ Pass if: **zero rows returned** (no duplicates from the (`ON CONFLICT DO NOTHING`)).

5. Cart

TC-11: Add a product+variant+size combo to cart

App steps: Product Catalogue → pick the product from TC-04 → pick variant/size 6 → Add to Cart.

sql

```
SELECT c.cart_id, c.vendor_id, c.product_id, cs.variant_id, s.size_value
FROM cart c
JOIN cart_size cs ON cs.cart_id = c.cart_id
JOIN size s ON s.size_id = cs.size_id
WHERE c.vendor_id = <id_from_TC01>;
```

✓ Pass if: one `cart` row for this vendor+product, one `cart_size` row for the chosen variant/size.

TC-12: Add another size for the same product (reuses the same cart row)

App steps: Add size 8 of the same variant to the cart.

sql

```
SELECT COUNT(DISTINCT cart_id) AS cart_rows, COUNT(*) AS size_rows
FROM cart c JOIN cart_size cs ON cs.cart_id = c.cart_id
WHERE c.vendor_id = <id_from_TC01> AND c.product_id = <id_from_TC04>;
```

✓ Pass if: `cart_rows = 1` (no duplicate cart entry for the same product), `size_rows = 2`.

TC-13: Cart screen shows correct aggregate counts

sql

```
SELECT c.cart_id, p.product_name,
       COUNT(DISTINCT v.variant_id) AS variant_count,
       ARRAY_AGG(DISTINCT s.size_value) AS selected_sizes
FROM cart c
JOIN product p ON c.product_id = p.product_id
LEFT JOIN variant v ON v.product_id = p.product_id
LEFT JOIN cart_size cs ON cs.cart_id = c.cart_id
LEFT JOIN size s ON s.size_id = cs.size_id
WHERE c.vendor_id = <id_from_TC01>
GROUP BY c.cart_id, p.product_name;
```

✓ Pass if: `selected_sizes` matches what the Cart screen displays.

TC-14: Remove a product from cart

sql

```
SELECT * FROM cart WHERE vendor_id = <id_from_TC01> AND product_id = <id_from_TC04>
```

✓ Pass after removing in-app: zero rows (and the corresponding `cart_size` rows are gone too via cascade/manual delete — check `cart_size` directly if no `ON DELETE CASCADE` is defined).

6. Hold List

TC-15: Move a product to Hold List

App steps: From Catalogue, hold a product instead of adding to cart.

sql

```
SELECT h.hold_id, h.vendor_id, h.product_id, p.product_name
FROM hold_list h JOIN product p ON p.product_id = h.product_id
WHERE h.vendor_id = <id_from_TC01>;
```

✓ Pass if: row exists.

TC-16: Hold List (global) shows items across all vendors

sql

```
SELECT h.hold_id, v.vendor_name, p.product_name
FROM hold_list h
JOIN product p ON h.product_id = p.product_id
JOIN vendor v ON h.vendor_id = v.vendor_id
ORDER BY h.created_date DESC;
```

✓ Pass if: matches what the Dashboard's global Hold List view shows, including vendor name.

7. Purchase Order (the critical multi-table transaction)

TC-17: Create a purchase order from cart

App steps: Re-add a product+variant+sizes to cart (TC-11/12) → Cart screen → Create Purchase Order → enter cost/MRP/qty/GST per size → Confirm.

sql

```

-- 1. PO header
SELECT po_id, vendor_id, order_date, status FROM purchase_order
WHERE vendor_id = <id_from_TC01> ORDER BY created_date DESC LIMIT 1;

-- 2. products on the PO
SELECT ppd.po_product_id, p.product_name
FROM po_product_detail ppd JOIN product p ON p.product_id = ppd.product_id
WHERE ppd.po_id = <po_id_above>;

-- 3. variants on the PO
SELECT pvd.po_variant_id, pvd.variant_id
FROM po_variant_detail pvd
WHERE pvd.po_id = <po_id_above>;

-- 4. size-level pricing (and confirm the generated 'amount' column computed corre
SELECT psd.size_id, s.size_value, psd.cost, psd.mrp, psd.qty, psd.gst, psd.amount
FROM po_size_detail psd JOIN size s ON s.size_id = psd.size_id
WHERE psd.po_id = <po_id_above>;

```

✅ Pass if:

- exactly one `purchase_order` row, `status = 'confirmed'`
- one `po_product_detail` row per distinct product ordered
- one `po_variant_detail` row per distinct variant ordered
- one `po_size_detail` row per size, and `amount = cost * qty` for every row (this is a generated column — if it doesn't match, something's wrong upstream, not in the DB)

TC-18: Placing an order clears the cart

sql

```
SELECT * FROM cart WHERE vendor_id = <id_from_TC01>;
```

✅ Pass if: zero rows — confirms `PurchaseOrder.create_order()` calls `Cart.clear_for_vendor()` after a successful transaction.

TC-19: Order total on the Orders list matches the sum of size details

App steps: Open Orders list (global or vendor-scoped).

sql

```
SELECT po.po_id, v.vendor_name, COALESCE(SUM(psd.amount), 0) AS order_total
FROM purchase_order po
JOIN vendor v ON po.vendor_id = v.vendor_id
LEFT JOIN po_size_detail psd ON psd.po_id = po.po_id
WHERE po.po_id = <po_id_from_TC17>
GROUP BY po.po_id, v.vendor_name;
```

✓ Pass if: `order_total` matches the amount shown in the app's Orders list for this PO.

TC-20: Order details screen shows full breakdown

sql

```
SELECT * FROM purchase_order WHERE po_id = <po_id_from_TC17>;

SELECT DISTINCT p.product_id, p.product_name
FROM po_product_detail ppd JOIN product p ON ppd.product_id = p.product_id
WHERE ppd.po_id = <po_id_from_TC17>;

SELECT psd.*, s.size_value
FROM po_size_detail psd JOIN size s ON psd.size_id = s.size_id
WHERE psd.po_id = <po_id_from_TC17>
ORDER BY s.size_value;
```

✓ Pass if: every field shown on the Order Details screen (products, sizes, cost, MRP, qty, GST, amount) matches these rows exactly.

TC-21: A failed order does not partially commit

App steps: Try to force a failure mid-order if possible (e.g. kill network mid-submit, or submit with an invalid size_id via API testing tool).

sql

```
SELECT * FROM purchase_order WHERE vendor_id = <id_from_TC01>
AND created_date > NOW() - INTERVAL '5 minutes';
```

✓ Pass if: **no partial PO** exists — `create_order()` wraps product/variant/size detail inserts in `execute_transaction`, so it should be all-or-nothing.

8. Roles & Permissions

TC-22: Roles list loads

App steps: Open Roles/Settings screen (if wired into the mobile UI) or hit `GET /api/roles` directly.

sql

```
SELECT role_id, role_name, is_system_role, status FROM role ORDER BY role_id;
```

✓ Pass if: matches the 5 seeded roles, in the same order the API returns.

TC-23: Role permissions reflect what was assigned

sql

```
SELECT r.role_name, p.module, p.action, p.permission_key
FROM role_permission rp
JOIN role r ON r.role_id = rp.role_id
JOIN permission p ON p.permission_id = rp.permission_id
WHERE r.role_id = 1
ORDER BY p.module, p.action;
```

✓ Pass if: Administrator (role_id 1) shows **all 57** permissions after running the reset procedure.

TC-24: Updating a role's permissions replaces, not appends

App steps (or via `PUT /api/roles/2/permissions`): Assign 3 permissions to Procurement Manager, save, then save again with a different set of 2 permissions.

sql

```
SELECT COUNT(*) FROM role_permission WHERE role_id = 2;
```

✓ Pass if: count is exactly `2` after the second save — not 5 (confirms `Role.set_permissions()` deletes the old set first, since it's documented as "not additive").

9. Cross-cutting checks (run after a full pass through the above)

TC-25: No orphaned rows

sql

```
-- variants pointing at a deleted product (shouldn't happen — FK enforced, but con
SELECT v.variant_id FROM variant v
LEFT JOIN product p ON v.product_id = p.product_id
WHERE p.product_id IS NULL;

-- po_size_detail pointing at a PO that doesn't exist
SELECT psd.po_size_id FROM po_size_detail psd
LEFT JOIN purchase_order po ON psd.po_id = po.po_id
WHERE po.po_id IS NULL;
```

✓ Pass if: both return zero rows.

TC-26: Sequence sanity after heavy testing

```
sql

SELECT 'vendor' AS tbl, MAX(vendor_id) AS max_id, (SELECT last_value FROM vendor_v
UNION ALL
SELECT 'product', MAX(product_id), (SELECT last_value FROM product_product_id_seq)
UNION ALL
SELECT 'purchase_order', MAX(po_id), (SELECT last_value FROM purchase_order_po_id_
```

✓ Pass if: `seq_val >= max_id` for every table — if a sequence falls behind its max id (can happen after mixing manual-ID inserts, like our reset procedure, with app inserts), the next app insert will throw a duplicate-key error.

Suggested run order for a full regression pass

1. `CALL reset_shinde_procurement_defaults();`
2. Section 1 → 2 → 3 → 4 (Vendor → Product → Variant → Sizes) — builds your test data
3. Section 5 → 6 (Cart / Hold List)
4. Section 7 (Purchase Order) — the highest-risk area, since it's the only multi-table transaction in the app
5. Section 8 (Roles) — independent of the above, can run anytime
6. Section 9 — final sanity pass